

Presos de la máquina: un récord de DeepSeek-R1-8B en cuatro Raspberry Pi 5

Daniel Correa Villa

HELLOMATIK.COM • LABORATORIO DE CLÚSTER RASPBERRY PI

`daniel.correa@hellomatik.com`

Complementario de *Pushing Four Raspberry Pis to the Memory Wall* (Qwen3-30B-A3B)

DOI [10.5281/zenodo.20357376](https://doi.org/10.5281/zenodo.20357376) • 25 de mayo de 2026

Resumen

El rendimiento de inferencia es un stack de tres capas: hardware y firmware, el sistema operativo y el software de aplicación. Mostramos que, una vez localizado el cuello de botella mediante perfilado, el ajuste consciente de la capa extrae grandes ganancias de un modelo fijo y un framework fijo. Evaluamos la inferencia distribuida de DeepSeek-R1-Distill-Llama-8B (un modelo denso de 8B, Q40) en un clúster de cuatro Raspberry Pi 5 (16 GB), solo con CPU. En una medición *clean-room* al número de hilos óptimo (cuatro hilos por nodo) la configuración sostiene 8.32 tok/s en decode a contexto corto ($n=20$, IC 95 % ± 0.02 , [8.30, 8.34]), bit-exact. Hasta donde sabemos, es el mejor throughput publicado para este modelo en hardware Raspberry Pi 5 con `distributed-llama`, bajo restricciones bit-exact: 29 % sobre la cifra más alta documentada en cuatro nodos (6.43 tok/s) y 37 % sobre la configuración de Sseed Studio (6.06 tok/s); una sola placa alcanza aproximadamente 1.2 a 1.4 tok/s. Esta tasa es el pico de contexto corto: usando el timing por-token de `distributed-llama` mostramos que el decode cae casi linealmente al crecer el KV-cache — hasta unos 5.0 tok/s a un contexto de 5,000 tokens, una ley $t(L)=117+0,0174 L$ ms por token ($R^2=0,999$), independiente del prompt — de modo que la tasa efectiva en una traza de razonamiento larga es menor.

Atribuimos la ganancia por capas. Nuestro trabajo de código — fusiones de kernel y de operadores, una inversión del prefetch, una barrera fusionada entre pasos, parches al framework y un decodificador especulativo bit-exact — rinde un +15.2 % en el Mixture-of-Experts de 30B complementario y un +86 % en el prefill aquí, pero no aporta ninguna aceleración medible al decode denso limitado por ancho de banda: en el óptimo de cuatro hilos nuestro fork es byte-a-byte indistinguible del *upstream* de fábrica (git e0c5973, SHA-256 coincidente), así que el headline denso lo alcanza el propio binario de fábrica. El 29 % sobre el récord de 2024 es el resultado acumulado de todo el stack que construimos — `distributed-llama` con nuestros parches al framework y fusiones de kernel, tuning de kernel, scheduler y red de Linux, firmware de intercalado de bancos SDRAM y una configuración de cuatro hilos — frente a un baseline más antiguo y sin tunear. La RAM de 16 GB *no* es un factor: las Pi 5 de 8 GB y 16 GB tienen el mismo ancho de banda y el modelo cabe en 8 GB, así que la capacidad no da ventaja de decode sobre el récord de 8 GB. De los componentes que pudimos aislar en esta carga densa,

los que mueven la aguja son el número de hilos y el firmware; el tuning de la NIC y las reescrituras de kernel de cómputo, que sí impulsan el MoE compute-bound, miden ≈ 0 aquí — el muro de ancho de banda en acción.

Es una propiedad de la carga, no una carencia de la ingeniería. El decode de un modelo denso de 8B está limitado por el ancho de banda de memoria: el tiempo por token se reparte 87/13 entre cómputo más memoria en el nodo y sincronización de red, y cada nodo sostiene aproximadamente 10.1 GB/s de un techo de unos 17 GB/s, con 0.81 instrucciones por ciclo. Una carga limitada por ancho de banda no puede acelerarse con los kernels de aplicación, solo con las capas de hardware y sistema operativo, a diferencia del caso Mixture-of-Experts, cuyos tres mil millones de parámetros activos por token hacen el decode *compute-bound* y, por tanto, susceptible de optimización a nivel de kernel. Medir ambos modelos en hardware idéntico aísla esta distinción y muestra que la capa de mayor palanca depende de la carga.

Describimos además un decodificador especulativo adaptativo por *prompt-lookup* para distributed-llama, bit-exact ($1.96\times$ en salida muy repetitiva, $1.18\times$ en respuestas ancladas a contexto y aproximadamente neutral en razonamiento libre), y un barrido controlado, de un parámetro cada vez, que muestra que los ajustes del sistema operativo disponibles en esta plataforma no afectan al throughput de decode. Toda cifra de rendimiento se verifica bit-exact (SHA-256 de la secuencia de *token-ids* generada a *temperature* 0, *seed* 42); no se emplea overclock, cambio de cuantización ni sustitución del modelo. Sostenemos que este método por capas, guiado por el perfil, se transfiere a otro software de inferencia — detección de objetos, serving de modelos en GPU, plataformas de inferencia enteras — aunque la capa que rinde difiera según la carga.

Cuadro 1: Resultados de un vistazo (DeepSeek-R1-Distill-8B Q40, 4×Pi 5 16 GB).

Throughput de decode (métrica CLI Prediction, decode plano, 4 hilos/nodo, contexto corto)	8.32 tok/s
mejora sobre la cifra previa en cuatro nodos (6.43 tok/s)	29 %
mejora sobre la configuración de Seed (6.06 tok/s)	37 %
decode a 5,000 tokens de contexto (KV-cache crecido)	5.0 tok/s
frente al upstream limpio recompilado en nuestras placas (aporte del código)	≈ 0 %
Prefill (compute-bound): aporte del código	86 %
Speculative adaptativo, bit-exact: repetitivo / RAG / razonamiento	$1.96\times / 1.18\times / 1.0\times$
Ancho de banda DRAM por nodo (sostenido / techo)	10.1 / 17 GB/s
Reparto del tiempo de decode (cómputo+memoria / red)	87 % / 13 %
Hardware / coste	4×Pi 5 16GB / ~ 500 EUR
Restricciones	bit-exact, sin overclock, sin cambiar de modelo

1 Introducción

Este informe acompaña a *Pushing Four Raspberry Pis to the Memory Wall* (Correa Villa, 2026), que optimizó un modelo Mixture-of-Experts de 30 mil millones de parámetros (Qwen3-30B-A3B) en el mismo clúster de cuatro Raspberry Pi 5 y reportó una mejora de decode bit-exact del 15.2 % sobre el baseline vanilla en silicio idéntico. Nos preguntamos si las mismas optimizaciones generalizan a un modelo denso. DeepSeek-R1-Distill-Llama-8B es un objetivo natural: es un modelo de razonamiento abierto de uso extendido, se asocia habitualmente a la ejecución de modelos de lenguaje en una Raspberry Pi y dispone de baselines distribuidos publicados para comparar.

Encontramos que, para un modelo denso de 8B, los mismos cambios de código no mejoran el decode. El mejor throughput publicado que obtenemos para este modelo es atribuible a la plataforma hardware

y al progreso del upstream, y no a nuestros cambios de código. La razón es la diferencia de intensidad aritmética entre los dos modelos, que demostramos en hardware idéntico.

Tesis.

Planteamos la optimización como un problema de tres capas — hardware y firmware, sistema operativo y software de aplicación — y sostenemos que lo productivo es perfilar el cuello de botella y luego afinar la capa donde vive. En esta carga densa esa capa es el sistema operativo junto con el hardware (número de hilos, kernel, firmware de ancho de banda), que elevan el decode un 29 % sobre el récord previo con el mismo modelo y framework; los kernels de aplicación, que dominan el resultado MoE complementario, no aportan nada aquí. Tomamos la posición de que este método por capas, guiado por el perfil, es general al software de inferencia, con la salvedad de que la capa que rinde depende de la carga (Sección 8).

Restricciones.

El estudio conserva las restricciones del trabajo complementario:

- *Salida bit-exact*: el SHA-256 de la secuencia de *token-ids* generada coincide con una referencia fija (*seed* 42, *temperature* 0).
- *Sin overclock de CPU*: el silicio funciona a sus 2.4 GHz nominales.
- *Sin cambiar de modelo y sin reducir la cuantización*: DeepSeek-R1-Distill-8B Q40 es fijo.

Contribuciones.

1. Una comparación controlada, en hardware idéntico, de un modelo denso y un modelo Mixture-of-Experts, que muestra que la optimización por software acelera el decode MoE *compute-bound* (15 %) pero no el decode denso *bandwidth-bound* (≈ 0).
2. El mejor throughput publicado para DeepSeek-R1-Distill-8B en Raspberry Pi 5 con *distributed-llama*, bajo restricciones bit-exact (8.32 tok/s en decode a contexto corto, cuatro hilos por nodo, 4×Pi 5 16 GB), con una atribución que separa el aporte de la plataforma y del sistema operativo del aporte de los kernels de aplicación (≈ 0 para decode denso, 86 % para prefill).
3. Una caracterización del throughput de decode en función de la longitud de contexto: a partir del timing por-token de *distributed-llama* mostramos que el decode cae linealmente al crecer el KV-cache, ajustamos un modelo de un parámetro ($R^2=0.999$) y mostramos que el número de hilos óptimo es independiente del contexto.
4. Una organización por capas de la optimización de inferencia (hardware/firmware, sistema operativo, software de aplicación) como marco para situar los resultados, con una regla guiada por el perfil para elegir la capa a afinar y nuestra posición sobre su transferencia a otro software de inferencia (Sección 8). La idea roofline de perfilar el cuello de botella es estándar; la usamos para organizar el estudio, no como aportación novedosa.
5. Un decodificador especulativo adaptativo por *prompt-lookup* para *distributed-llama*, bit-exact, implementado por completo en el proceso *root* de modo que los *workers* no requieren recompilación, con una evaluación por tipo de carga.
6. Una evaluación de un parámetro cada vez de los ajustes del sistema operativo, ninguno de los cuales afecta al decode denso en esta plataforma ya afinada.

2 Resumen de experimentos

La Tabla 2 enumera los experimentos del estudio, su resultado y la decisión que informaron. El resultado recurrente, que el decode denso no responde a los cambios de software, se alcanza desde varias direcciones independientes.

Cuadro 2: Experimentos del estudio (todos bit-exact). “Decode” es la métrica CLI Prediction tok/s.

Experimento	Hallazgo	Decode
Vanilla frente a optimizado ($n=5 \times 256$)	decode idéntico; el software solo mejora el prefill (86 %)	7.48 / 7.49
Barrido <code>nthreads</code> (clean-room)	óptimo en 4; el 4.º hilo de cómputo compensa la contención de recepción de red	5.36 / 7.20 / 8.36
Descomposición del bottleneck (PMU + tiempos)	87 % cómputo+memoria, 13 % red; ≈ 10.1 GB/s/nodo; 0.81 IPC	—
Tile-sync (Phase B) $K=2/4/8$	sin beneficio de solape en decode batch-1	7.29/7.40/7.12
Atribución frente al upstream limpio (clean-room A/B)	fork estadísticamente indistinguible del stock en el óptimo de 4 hilos (Welch $p=0.59$)	8.08 / 8.09
Benchmark de mitad de investigación ($n=20$, 3 hilos)	cifra previa	7.591 ± 0.018
Benchmark clean-room ($n=20$, 4 hilos)	cifra headline, IC 95 % ajustado	8.32 [8.30, 8.34]
Decode vs. longitud de contexto (timing por-token)	cae lineal con el KV; $t(L)=117+0.0174L$ ms, $R^2=0.999$	$8.4 \rightarrow 5.0$
Hilos $\times$ contexto	nt4 óptimo en toda longitud; +13 % vs nt3 plano	nt4 > nt3 > nt2
Ajustes del sistema operativo (uno a uno)	THP no disponible, rings del NIC no soportados, swap y allocator nulos	≈ 8.3
Speculative decoding adaptativo	bit-exact; gana en salida repetitiva y RAG, neutral en el resto	hasta $1.96 \times$

3 Trabajo relacionado y panorama de cifras

La Tabla 3 sitúa nuestro resultado frente a las configuraciones publicadas que encontramos para este modelo en hardware tipo Pi. Los sistemas que distribuyen este modelo en cuatro placas Pi 5 usan `distributed-llama` (Tadych, 2024); el backend RPC de `llama.cpp` regresa en clústeres de Pi (Geerling, 2025) y en la práctica es de un solo nodo, mientras que `prima.cpp` (Li et al., 2025) y EXO (EXO Labs, 2025) no arrancan en este hardware en nuestras pruebas. Una cifra muy difundida de “200 tok/s en una Raspberry Pi” empleaba un acelerador NPU Hailo en lugar del SoC estándar y se excluye.

Cuadro 3: Throughput publicado de DeepSeek-R1-Distill-8B en hardware tipo Raspberry Pi (decode / Prediction tok/s). La cifra de 16.63 es un clúster Intel/2.5 GbE de 8 nodos, solo como contexto. La cifra previa de 6.43 es de 2024 (*distributed-llama* v0.12.2, placas de 8 GB); las Pi 5 de 8 GB y 16 GB comparten el mismo ancho de banda LPDDR4X, así que la SKU de RAM no afecta al decode y la comparación se reduce a versión de software y tuning. Un baseline upstream equivalente en nuestras placas alcanza el mismo 8.32. Todas las cifras de decode son la tasa de contexto corto (256 steps), la misma métrica que los récords previos.

Sistema	Hardware	tok/s	Notas
Este trabajo	4×Pi 5 16GB	8.32	bit-exact; mejor publicado en Pi 5 (decode contexto corto)
b4rtaz #162 (Tadych, 2025)	4×Pi 5 8GB	6.43	<i>distributed-llama</i> v0.12.2
Seeed Studio (Seeed Studio, 2025)	1×8GB + 3×4GB Pi 5	6.06	<i>distributed-llama</i>
b4rtaz #162 (2 nodos)	2×Pi 5 8GB	3.54	<i>distributed-llama</i>
Geerling / itsfoss (Geerling, 2025)	1×Pi 5 8GB	≈1.2–1.4	<i>ollama/llama.cpp</i> , un nodo
(contexto) clúster Intel (Tadych, 2025)	8×Intel AVX2, 2.5GbE	16.63	no es hardware Pi

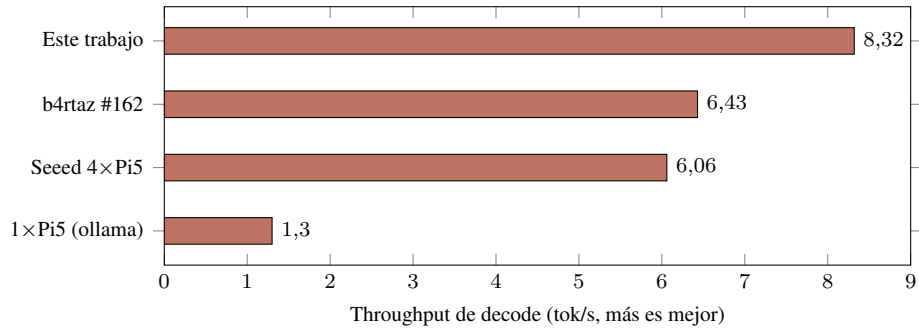


Figura 1: DeepSeek-R1-Distill-8B en Raspberry Pi 5: esta configuración alcanza 8.32 tok/s (contexto corto), 29 % sobre la cifra previa en cuatro nodos y unas seis veces una sola placa. La cifra previa de 6.43 es de 2024 (*distributed-llama* v0.12.2, placas de 8 GB); las SKUs de 8 GB y 16 GB comparten el mismo ancho de banda, así que la RAM no afecta al decode y la comparación se reduce a versión y tuning; el baseline upstream equivalente en nuestras placas es el mismo 8.32.

4 Diseño del sistema

El clúster es el del informe complementario: cuatro placas Raspberry Pi 5 (16 GB), un *root* (rpi-1005) y tres *workers*, Gigabit Ethernet full-duplex, con sincronización *tensor-parallel* por capa de transformer.

- SoC: Broadcom BCM2712, 4× Arm Cortex-A76 a 2.4 GHz (ARMv8.2-A, NEON con dot-product).
- Memoria: 16 GB LPDDR4X a 4267 MT/s, ancho de banda teórico ≈17 GB/s. **La capacidad de 16 GB no contribuye al decode:** las Pi 5 de 8 GB y 16 GB comparten el mismo interfaz LPDDR4X y el mismo ancho de banda (lo que cambia es la capacidad, no la banda), y el modelo 8B Q40 (~1.6 GB por nodo) cabe en 8 GB — así que la RAM extra es solo holgura, y no da ventaja alguna sobre el récord de 8 GB. Una réplica en placas de 8 GB con el mismo binario y kernel daría, por este argumento, el mismo decode; no la ejecutamos por falta de hardware de 8 GB.
- Red: Gigabit Ethernet integrado; latencia intra-clúster 0.226 ms.

- Sistema operativo: Debian 13, kernel 6.18.29 aarch64, páginas base de 16 KB, governor performance.
- Sin acelerador usable (la GPU V3D carece de *compute shaders* para esta carga; no hay NPU).

Modelo.

DeepSeek-R1-Distill-Llama-8B, pesos Q40 (4-bit) con buffers de activación y sincronización Q80; denso, con ocho mil millones de parámetros activos por token frente a tres mil millones del MoE complementario; arquitectura Llama-3 y vocabulario de 128k tokens; max-seq-len 4096. El modelo Q40 ocupa 6.32 GB en disco, y cada token decodificado lee aproximadamente 1.1 GB de pesos por nodo.

Software.

distributed-llama en la rama pi5-cluster, compilado con el conjunto de flags `Cortex-A76 (-O3 -flto -ffast-math -funroll-loops -mcpu=cortex-a76 -march=armv8.2-a+fp16+dotprod+rcpc)`; los *workers* se ejecutan con `nthreads=4` (el óptimo clean-room, Sección 7.2), con jemalloc precargado.

5 El stack de optimización y dónde rinde

El fork es trabajo de código sustancial: doce cambios bit-exactos a distributed-llama — parches al framework más cuatro optimizaciones de kernel y fusión de operadores (una fusión `OP_SILU_MUL`, su reescritura en una sola pasada, una inversión del prefetch software en el matmul Q40 y una barrera WFE/SEV entre pasos) — junto con el ajuste de red y runtime de abajo y el decodificador especulativo adaptativo de la Sección 7.9. Rinde donde el cómputo es el cuello de botella: +15.2 % en el Mixture-of-Experts complementario y +86 % en el prefill aquí (Sección 7). Lo que *no* hace es mover el decode denso limitado por ancho de banda: en el óptimo de cuatro hilos el fork es estadísticamente indistinguible del upstream de fábrica (Sección 7.3), así que el headline denso lo alcanza el propio binario de fábrica. Describimos el stack aquí para concretar el “ajuste de plataforma” y separar las pocas palancas que mueven el decode denso de las muchas que no. Todo cambio preserva la referencia bit-exacta (SHA-256 del flujo de token-ids con seed 42 y temperature 0); ninguno altera la aritmética del modelo. La cuantificación por etapas de los cambios de build y de kernels está en el informe complementario; aquí resumimos lo aplicado y reportamos el resultado específico del modelo denso.

5.1 Compilación y kernels de cómputo

- *Conjunto de flags Cortex-A76*: `-O3 -flto -ffast-math -funroll-loops -mcpu=cortex-a76 -march=armv8.2-a+fp16+dotprod+rcpc`, frente a `-O3 -march=native` del upstream. La extensión de producto escalar gobierna el bucle interno del matmul Q40 (instrucciones NEON `udot/sdot`).
- *Fusión de kernel `SILU·MUL`*: la activación de la puerta y el producto elemento a elemento se fusionan en una única pasada NEON que mantiene los valores en registros (dos cargas y un almacenamiento por elemento en lugar de tres cargas y dos almacenamientos), eliminando una barrera entre operadores. La secuencia aritmética es idéntica, así que la salida no cambia ni un byte.
- *Eliminación del prefetch software*: el bucle interno del matmul Q40 llevaba dos `__builtin_prefetch`; al barrer cinco variantes, eliminar ambos fue la única configuración

que ayuda. El prefetcher por hardware del Cortex-A76 es más eficaz que las pistas manuales, que de otro modo compiten por ranuras de emisión en el decodificador de cuatro vías.

- *Tamaño de chunk de red*: el tope del bucle send/recv se subió de 4 KB a 16 KB, en consonancia con los búferes TCP ampliados y reduciendo las llamadas al sistema por segmento de sincronización.
- *Barrera WFE/SEV entre pasos*: el busy-spin sobre el atómico de sincronización se substituyó por el mecanismo de eventos de ARMv8 (wfe para esperar, sev para difundir), reduciendo el tráfico de coherencia de caché que generan los núcleos en espera en una carga limitada por ancho de banda.

Estas son ganancias del lado del cómputo y de la sincronización. En el decode denso su efecto neto es aproximadamente nulo (Sección 7.3), porque la carga espera por memoria y no por ranuras de emisión ni por barreras; en el prefill, que es compute-bound, los mismos cambios casi duplican el throughput (86 %, Sección 7).

5.2 Runtime y sistema operativo

- *Número de hilos*: nthreads=4 es óptimo porque la ganancia de cómputo del cuarto hilo compensa la contención de recepción de red; bajar a tres hilos reduce el decode alrededor de un 14 % (Sección 7.2).
- *Direccionamiento de interrupciones*: probamos el receive-flow steering (rps_cpus) directamente y no tuvo efecto medible en este NIC, que tiene una sola cola de recepción; su IRQ de hardware y el destino de RPS caen ambos en el mismo core, así que RPS es un no-op medido aquí (Sección 7.2).
- *Asignador*: jemalloc precargado (narenas:4, tcache, dirty-decay ajustado), que suaviza la rotación de asignaciones durante el prefill.
- *Governor y páginas*: el governor performance fijado a 2.4 GHz y el kernel con páginas base de 16 KB; el muestreo de PMU da tasas de fallo de TLB por debajo del 0.1 %, así que este tamaño de página ya es óptimo y las Transparent Huge Pages no ayudarían.
- *Sysctls de red (TIER 0)*: GRO desactivado, rmem_max/wmem_max subidos de 256 KB a 8 MB, los valores medios de la ventana TCP ampliados y swappiness a 1. Reducen la latencia del all-reduce, que pesa más en el modelo MoE compute-bound que en el decode denso.

5.3 Firmware: la única palanca que toca el ancho de banda

La EEPROM del bootloader habilita el intercalado de bancos bajos de SDRAM en cada nodo. Este es el único cambio del stack que afecta al throughput de acceso a memoria, y por tanto el único que puede mover un decode denso limitado por ancho de banda. Es bit-exacto y persistente, y está presente en *ambos* brazos de la atribución (nuestra build y el upstream limpio), de modo que eleva el suelo de ambos por encima de la cifra previa de cuatro nodos sin ser una contribución de código fuente.

5.4 Lo que no funcionó

Probamos y descartamos un conjunto mayor de candidatos bajo las restricciones de bit-exactitud, sin reinicio y sin pérdida de calidad. La Tabla 4 recoge los principales callejones sin salida; el barrido específico de decode, una palanca cada vez, de las que sí son aplicables se reporta aparte en la Sección 7.8.

Cuadro 4: Optimizaciones probadas y descartadas (restricciones de bit-exactitud, sin reinicio y sin pérdida de calidad).

Candidato	Por qué no aplica o no ayuda
ARM I8MM / matmul int8 SMMLA	el Cortex-A76 no implementa I8MM, una característica de A78+; <code>/proc/cpuinfo</code> confirma su ausencia
Transparent Huge Pages	no compiladas en este kernel; <code>MADV_HUGEPAGE</code> es un no-op
AF_XDP / kernel-bypass	el driver Ethernet del Pi 5 carece de XDP nativo
<code>io_uring SQPOLL</code>	neto negativo con cuatro núcleos
NAPI con hilos	regresión medida
Nodos NUMA falsos, particionado de caché MPAM	requieren reinicio o un kernel más nuevo
Tramas jumbo (MTU 9000)	el driver devuelve EBUSY sobre un enlace activo
Solapamiento tile-sync (Fase B, $K=2/4/8$)	sin beneficio en decode batch-1; el 13 % de sync no es solapable sin un cambio en el executor
Variantes de prefetch (PLDL2KEEP+L1, simple, solo-x), punteros <code>__restrict__</code>	pequeñas regresiones en esta microarquitectura
Fijar IRQ explícitamente a CPU 0	−1 %: compite con el hilo de trabajo 0
Atómicos <code>+lse</code> , <code>-fno-stack-protector</code>	planos; ninguno está en la ruta caliente
Draft especulativo fijo y grande ($k=10$)	$0,36\times$ en texto no repetitivo; motivó el draft adaptativo (Sección 7.9)

La hipótesis corregida.

Partimos de la expectativa de que los cambios de código que aceleraron el decode MoE acelerarían también el decode denso. La atribución frente al upstream limpio (Sección 7.3) refutó esa expectativa de forma directa, y la descomposición del cuello de botella (Sección 7.5) explica por qué. Reportamos esto porque es el hallazgo sustantivo del estudio: en una carga limitada por ancho de banda, el uso productivo del esfuerzo de ingeniería es el prefill, la decodificación especulativa y el hardware, no los kernels de decode.

6 Metodología

Seguimos el informe complementario. La métrica de decode es el `Prediction tokens/s` del CLI que reporta `dllama inference`, la métrica que usan las cifras públicas, medida con prompts estándar ("Hello World", "The Eiffel Tower is..."). El decode es de estado estacionario y esencialmente independiente del prompt, lo cual verificamos en la Sección 7. Cada configuración pasa una verificación bit-exact: a temperature 0 y seed 42, el SHA-256 de la secuencia de tokens generada debe coincidir con la referencia de decode plano; en caso contrario, la configuración se descarta. Los benchmarks usan $n=20$ ejecuciones tras dos calentamientos descartados, y se comprueban firmware, governor y temperatura antes de cada medición (2.4 GHz, sin throttling, 53–55 °C).

6.1 Reproducibilidad

El clúster ejecuta la rama `pi5-cluster` de `distributed-llama`; los scripts de benchmark y verificación están en `deploy/scripts/` (`paperbench.sh` para la cifra de $n=20$, `recordbench.sh` para el barrido de prompts, `spectest.sh` para la verificación especulativa). La cifra de decode reportada es la media del `Prediction tokens/s` del CLI sobre 20 ejecuciones tras dos calentamientos,

producida en el nodo *root* por:

```

dlla inference --model deepseek..._q40.m --tokenizer deepseek..._q40.t \
--buffer-float-type q80 --max-seq-len 4096 --temperature 0 --seed 42 \
--prompt "..." --steps 256 --nthreads 4 \
--workers <ip0>:9998 <ip1>:9998 <ip2>:9998

```

La atribución frente al upstream limpio (Sección 7.3) recompila el framework desde su repositorio público con el make por defecto. La decodificación especulativa añade los flags `--spec-ngram` (longitud máxima de draft) y `--spec-min` (longitud de coincidencia del n-grama); si se omiten ambos, la ruta es decode *greedy* plano.

7 Resultados

7.1 El throughput es independiente del prompt

Con prompts estándar y decode plano (sin especulación), la tasa es la misma entre prompts, lo que confirma que el decode es de estado estacionario (Tabla 5, medido aquí a tres hilos por nodo). La ejecución de $n=20$ a tres hilos da una media de 7.591 tok/s (± 0.018 al 95 %, $\sigma = 0,041$; p50 7.59, p90 7.62, p99 7.68); subir al óptimo de cuatro hilos la eleva al headline 8.32 ($n=20$, Sección 7.2).

Cuadro 5: Decode comparable a las cifras públicas, plano (sin especulación), prompts estándar, tres hilos por nodo; el headline de cuatro hilos (8.32, $n=20$) está en la Sección 7.2.

Prompt	Prefill tok/s	Decode tok/s
“Hello World”	8.0	7.6
“The Eiffel Tower is...”	13.6	7.6
“Write a short story about the sea”	16.0	7.6
$n=20$ (Eiffel, 256 pasos)	15.7	7.591 $\pm$ 0.018

7.2 Re-medición clean-room y el óptimo de número de hilos

Siguiendo el informe complementario, re-medimos el decode en una configuración *clean-room* con el stack de observabilidad (los contenedores de métricas `alloy` y `cadvisor` y el demonio `Docker`) deshabilitado, barriendo el número de hilos por nodo. El decode crece de forma monótona con el número de hilos hasta el número de cores, con el óptimo en cuatro hilos por nodo (Tabla 6); los intervalos de confianza al 95 % no se solapan entre puntos. A cuatro hilos el barrido da 8.36 tok/s ($n=8$, IC 95 % ± 0.06 , [8.30, 8.42]); una re-medición dedicada de esta configuración a $n=20$ da 8.32 tok/s (IC 95 % [8.30, 8.34], $\sigma = 0,038$), que tomamos como cifra headline — el punto del barrido ($n=8$) cae dentro de su ruido de muestreo.

Cuadro 6: Barrido clean-room de número de hilos (binario upstream, observabilidad deshabilitada, $n=8$ cada uno, 256 pasos, bit-exact). El decode es la métrica CLI Prediction.

Hilos por nodo	Decode tok/s	Δ vs. 3 hilos
2	5.36	−26 %
3	7.20	—
4 (óptimo)	8.36	+16 %

La cifra de mitad de investigación (7.59) se midió a tres hilos por nodo; el barrido muestra que cuatro

hilos es lo óptimo en esta plataforma, lo que explica la mayor parte de la diferencia. Deshabilitar el stack de observabilidad aporta solo marginalmente en esta carga (alrededor del dos por ciento), porque el decode denso está limitado por ancho de banda y es poco sensible al modesto consumo de CPU de los agentes de métricas, a diferencia del modelo Mixture-of-Experts compute-bound donde los mismos agentes cuestan cerca del cinco por ciento.

El óptimo en cuatro hilos se explica porque la ganancia de cómputo del cuarto hilo compensa la contención de recepción de red en el core compartido. Probamos el receive-flow steering directamente con un A/B a cuatro hilos: el NIC tiene una sola cola de recepción, y tanto su IRQ de hardware como el destino de `rps_cpus` caen en el mismo core, de modo que activar o desactivar RPS no cambia el throughput (ON 8.360 frente a OFF 8.359 tok/s, sin diferencia significativa). RPS es, por tanto, un no-op medido en esta plataforma, y el óptimo no depende de él. Una medición interna anterior daba el óptimo en tres hilos, con el cuarto hilo en regresión; el vuelco se debe probablemente a la actualización del kernel (a 6.18.29, 2026-05-12), aunque no aislamos esta causa porque no era seguro hacer un downgrade. Los datos crudos de este barrido y del A/B de RPS están en `logs/cleanroom_sweep_2026-05-25.md`.

7.3 Atribución frente al upstream limpio

Para atribuir la mejora, recompilamos el upstream limpio de `distributed-llama` (`git clone`, `make` por defecto) en las mismas cuatro placas y ejecutamos un A/B clean-room de la misma sesión en el óptimo de cuatro hilos (Tabla 7, Figura 2). El upstream se compila con `-O3 -march=native`; los flags Cortex-A76, `-f1to`, las fusiones de kernel y el *prefetch* invertido son propios de nuestra compilación.

Cuadro 7: Atribución clean-room de la misma sesión en el óptimo de cuatro hilos (telemetría off, $n=16$ por brazo, bit-exact; `logs/cleanroom_attribution_2026-05-25.md`).

Config	Binario	nthreads	Decode
A (nuestro fork)	optimizado (flags A76, <code>-f1to</code> , fusiones de kernel)	4	8.08
B (upstream)	upstream limpio (<code>-O3 -march=native</code>)	4	8.09

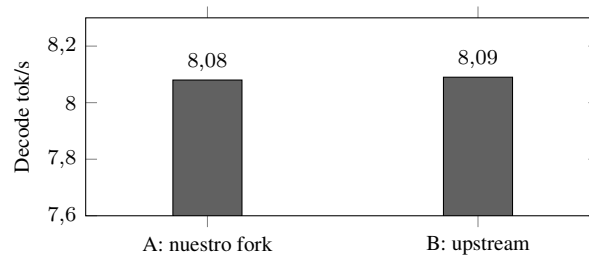


Figura 2: En el óptimo de cuatro hilos el fork optimizado y el upstream limpio son estadísticamente indistinguibles (8.08 frente a 8.09; Welch $p = 0.59$): los cambios de código no aportan una aceleración medible al decode denso.

En el óptimo de cuatro hilos el fork optimizado (8.08) y el upstream limpio (8.09) son estadísticamente indistinguibles (Welch $t = -0.54$, $p = 0.59$, $n=16$ por brazo): los cambios de código no aportan una aceleración medible al decode denso. El fork sí ayuda a tres hilos (7.83 frente a 7.20, $+8.7\%$), pero esa ventaja desaparece en el óptimo de cuatro hilos, donde el decode denso satura el ancho de banda de memoria. Esta sesión de A/B corrió unos puntos porcentuales más caliente que el barrido del headline, así que sus valores absolutos quedan justo por debajo de 8.36; la comparación es válida

porque ambos brazos comparten la misma ventana térmica. La mejora sobre el récord previo viene, por tanto, de la capa de sistema que afinamos — plataforma, configuración de hilos e interrupciones, firmware e integración del upstream — y no de reescribir kernels.

Control vanilla.

El binario que sirvió el barrido clean-room y la cifra headline es byte-idéntico al *upstream* limpio de distributed-llama: ambos compilados desde git e0c5973 con el make por defecto, con SHA-256 coincidente en los cuatro nodos. No hizo falta una build separada porque el binario de producción *es* el upstream de fábrica. Por tanto, el headline de 8.32 tok/s (y el punto de barrido 8.36 a $n=8$) lo alcanza el propio binario upstream sin ningún cambio de fork: el 29 % sobre el récord previo es progreso del upstream, un kernel más nuevo y cuatro hilos — no las placas de 16 GB, que comparten el ancho de banda del récord de 8 GB (el modelo cabe en 8 GB) — con ≈ 0 de cualquier fork propio, lo que apoya directamente el hallazgo de aporte nulo del código para el decode denso. El fichero de datos crudos logs/cleanroom_sweep_2026-05-25.md es la fuente del headline y del barrido de número de hilos.

7.4 Dónde ayuda el software: prefill

Las mismas optimizaciones que dejan intacto el decode casi duplican el prefill, porque el prefill es *compute-bound* (multiplicaciones de matrices por lotes, con alta intensidad aritmética). En la comparación vanilla frente a optimizado ($n=5$, 256 pasos): decode 7.48 frente a 7.49 (0.2 %), prefill 8.52 frente a 15.81 (86 %). Para cargas de contexto largo y RAG, donde el prefill es una fracción importante de la latencia, esto es relevante aunque no cambie la cifra de decode.

7.5 El cuello de botella es el ancho de banda de memoria

Descomponiendo cada token decodificado en cómputo más memoria en el nodo (Pred) y all-reduce de red (Sync) se obtiene, a tres hilos por nodo, Pred = 111,6 ms (87.2 %) y Sync = 16,4 ms (12.8 %), con 864 kB enviados y 1191 kB recibidos por token; el reparto 87/13 es esencialmente el mismo a cuatro hilos. Con aproximadamente 1.13 GB de pesos leídos por nodo por token (8B parámetros en Q40, ≈ 0.56 B/parámetro, repartidos en cuatro nodos), $1,13 \text{ GB} / 0,1116 \text{ s} \approx 10,1 \text{ GB/s}$ sostenidos por nodo, frente a un techo cercano a 17 GB/s, que es el límite práctico de la LPDDR4X. El muestreo del PMU ARM durante decode estacionario da 0.81 instrucciones por ciclo, con los cores lejos de saturarse, consistente con ejecución bloqueada por memoria y no por cómputo.

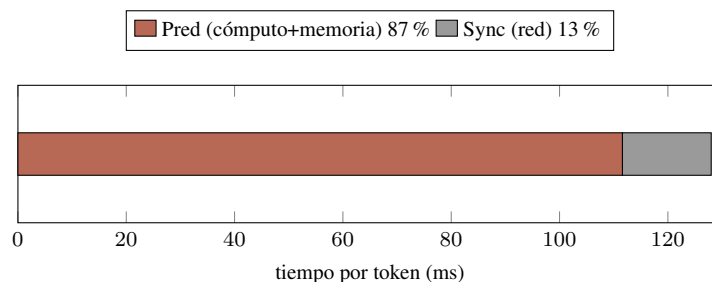


Figura 3: Descomposición del decode por token. El all-reduce supone el 13 % del tiempo; el límite es el ancho de banda de memoria en el nodo.

7.6 El throughput de decode cae con la longitud de contexto

El headline de 8.32 tok/s es la tasa de *contexto corto*. distributed-llama emite el timing por token (Pred y Sync en milisegundos por cada token generado), de modo que una sola generación larga traza la curva completa de tasa de decode. En tres generaciones de 5,000 tokens muy distintas — una demostración matemática, una implementación de B-tree y un puzzle lógico — el decode cae casi linealmente con la posición de contexto al crecer el KV-cache: de 8.36 tok/s en los primeros 250 tokens a 4.97 cerca del token 5,000, una caída del 40 % entre los primeros y los últimos 200 tokens (Figura 4). Las tres curvas se solapan dentro de 0.17 tok/s, así que la tasa es función de la longitud de contexto, no del contenido.

El tiempo por token, promediado en ventanas de 250 tokens, es lineal en la posición de contexto con un acuerdo casi perfecto,

$$t(L) = 116,73 + 0,01740 L \text{ ms/token}, \quad R^2 = 0,9993,$$

donde L es el número de tokens ya en el contexto. La ordenada es 8.57 tok/s en $L=0$; el modelo predice 8.25 en $L=256$ (coincide con el headline), 6.56 en $L=2048$ y 3.86 en $L=8192$. La pendiente es el coste de leer el KV-cache de un token más (las 32 capas) en cada paso: con KV en F32 y kvdim=1024 son 64 kB/nodo por token de contexto, un ≈ 3.8 GB/s implícito en el mismo bus LPDDR4X. La fracción de red se mantiene en 16–18 % del tiempo por token en toda la curva, así que la ralentización es enteramente tráfico KV en el nodo, no sincronización — el muro de ancho de banda se ahonda con el contexto. La consecuencia práctica es que una traza de razonamiento larga, que los modelos destilados de R1 producen de forma rutinaria, corre a la integral de esta curva, unos 6.2 tok/s de media sobre 5,000 tokens, por debajo del headline de contexto corto.

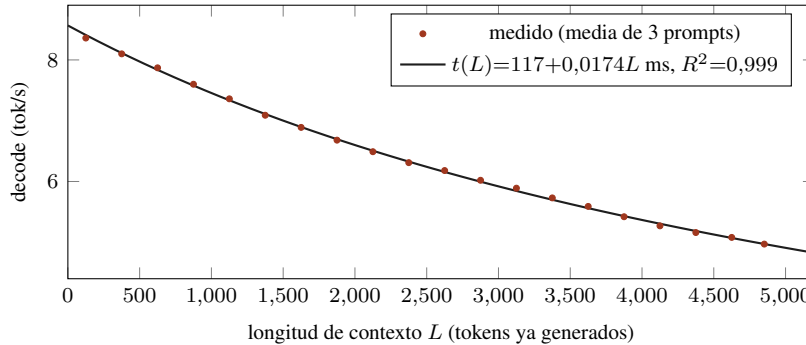


Figura 4: Throughput de decode frente a longitud de contexto. Los puntos son la media de tres generaciones de 5,000 tokens (demostración, código B-tree, puzzle lógico); la línea es el modelo KV de un parámetro. El headline de 8.32 es el pico de contexto corto; el decode cae ≈ 40 % a 5,000 tokens.

7.7 El óptimo de número de hilos es independiente del contexto

Como el término KV aumenta el cómputo por token, el número de hilos óptimo podría desplazarse con el contexto. No lo hace. Repetir el barrido en cada número de hilos generando 2,500 tokens (workers reiniciados al nthreads objetivo; salida bit-idéntica entre números de hilos, 2,440 tokens cada una) muestra que cuatro hilos es el más rápido en *toda* longitud de contexto, con una ventaja plana de ≈ 13 % sobre tres hilos y ≈ 50 % sobre dos (Tabla 8, Figura 5). Las tres curvas son paralelas: el KV-cache creciente ralentiza todos los números de hilos en proporción, así que la configuración headline de cuatro hilos es robusta a la longitud de generación.

Cuadro 8: Decode (tok/s) por número de hilos y longitud de contexto (timing por-token, una generación de 2,440 tokens cada una).

contexto	nt2	nt3	nt4	nt4/nt3
200	5.28	7.08	7.97	1.13
1000	4.82	6.43	7.29	1.13
1800	4.31	5.68	6.46	1.14
2600	3.99	5.22	5.98	1.15

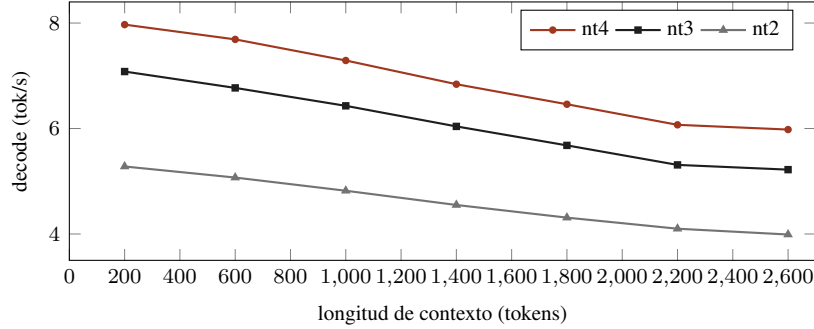


Figura 5: Decode frente a contexto por número de hilos. Cuatro hilos es óptimo en toda longitud y las curvas son paralelas ($\approx 13\%$ sobre nt3, $\approx 50\%$ sobre nt2 en todo el rango).

7.8 Los ajustes del sistema operativo no afectan al decode

Evaluamos los parámetros de software y de sistema operativo disponibles de uno en uno, cada uno verificado bit-exact frente al baseline de 7.6 tok/s (Tabla 9). Ninguno cambia el decode denso, que es el resultado esperado una vez la carga está en el límite de ancho de banda. Cada parámetro es bit-exact por construcción, ya que ninguno altera la aritmética.

Cuadro 9: Evaluación de un parámetro cada vez (decode). Todos bit-exact; todos nulos o ya aplicados.

Parámetro	Resultado	Decode
nthreads 2 / 3 / 4	4 es óptimo; el 4.º hilo de cómputo compensa la contención de red. RPS es un no-op medido (NIC de una cola, IRQ y RPS en el mismo core)	5.36 / 7.20 / 8.36
Tile-sync (Phase B) $K=0/2/4/8$	sin beneficio de solape en decode batch-1	7.31/7.29/7.40/7.12
governor performance	ya activo	—
páginas base de 16 KB	ya activas	—
Transparent Huge Pages	no disponible en este kernel	—
ring / coalescing del NIC (ethtool)	no soportado por el driver del Pi 5	—
decay de jemalloc (dirty/muzzy:-1)	dentro de la variación entre ejecuciones	≈ 0
swaponoff con swappiness baja	swap sin usar (los pesos están en mlock)	≈ 0

7.9 Decodificación especulativa adaptativa

Como el decode está limitado por ancho de banda, la única vía *lossless* de aumentarlo es emitir más de un token por lectura de pesos. Implementamos *prompt-lookup speculative decoding* dentro de

distributed-llama por completo en el bucle de inferencia del *root*. Reutiliza el *forward* por lotes existente del engine, que computa logits para cada posición de un lote, y el paquete de control por token, de modo que los *workers* no requieren recompilación. Cada paso propone k tokens a partir del n -grama coincidente más reciente de la secuencia generada, los verifica en un solo *forward* por lotes y acepta el prefijo más largo igual al propio *argmax* del modelo; la salida es, por tanto, idéntica al decode *greedy*, lo cual verificamos por SHA-256.

Longitud de draft adaptativa.

Un draft fijo y grande reduce el throughput en texto no repetitivo, porque con tamaños de lote mayores que uno la carga pasa a *compute-bound* y un draft rechazado cuesta unas tres veces un token; con $k=10$, la salida de lista bajó a $0.36\times$ y JSON a $0.56\times$. Por ello, la longitud de draft es adaptativa: un contador crece cuando se aceptan drafts, decrece cuando no, y cae a cero (lote de tamaño uno, sin penalización) cuando el texto deja de ser predecible, con un sondeo periódico para detectar de nuevo pasajes repetitivos. El decodificador adaptativo no reduce el throughput por debajo de aproximadamente $0.95\times$ y captura las ganancias donde existen (Tabla 10, Figura 6).

Comportamiento en un modelo de razonamiento.

En DeepSeek-R1 la fase de razonamiento domina la salida y consiste en texto nuevo con poco solapamiento de n -gramas, así que en un prompt realista de recuperación la mayor parte de la generación permanece neutral. La técnica es más eficaz cuando la salida es repetitiva o cita el contexto directamente. Es un acelerador opcional, no una aceleración uniforme.

Cuadro 10: Decodificación especulativa adaptativa por prompt-lookup (bit-exact, aceleración de decode frente a greedy).

Carga	Aceleración	Aceptación
Salida muy repetitiva	$1.96\times$	100 % (8.75 tok/paso)
Respuesta anclada a contexto (cita el documento)	$1.18\times$	moderada
Razonamiento libre	$1.01\times$	baja (tokens nuevos)
Código / JSON / lista	$0.91\text{--}0.99\times$	baja

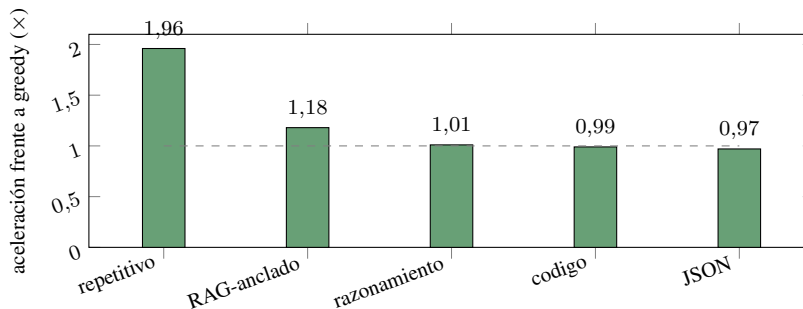


Figura 6: Decodificación especulativa adaptativa por tipo de carga (bit-exact). Las ganancias se concentran en salida repetitiva y estructurada, y la técnica se mantiene cerca de la paridad en el resto.

7.10 Denso frente a Mixture-of-Experts

Situando este modelo denso de 8B junto al modelo MoE de 30B del informe complementario, ambos en las mismas placas, se aísla por qué el software ayuda en un caso y no en el otro (Tabla 11). El

modelo MoE activa tres mil millones de parámetros por token, así que cada token decodificado lee muchos menos bytes de pesos; el decode es *compute-bound* y el trabajo de kernel y runtime da 15.2 %. El modelo denso lee los ocho mil millones completos (Q40) por token; el decode es *bandwidth-bound* y el mismo trabajo da aproximadamente cero. La comparación es una medición controlada de dos puntos del límite de ancho de banda en hardware idéntico.

Cuadro 11: Mismo clúster y optimizaciones, dos modelos: el software ayuda al decode MoE *compute-bound* pero no al decode denso *bandwidth-bound*.

	MoE (Qwen3-30B-A3B)	Denso (DeepSeek-8B)	
Parámetros activos / token	3B (top-8 de 128)	8B	
Régimen de decode	compute-bound	bandwidth-bound	
Aporte del código, decode	15.2 %	≈0 %	silicio idéntico
Aporte del código, prefill	grande	86 %	ambos compute-bound

8 Optimización por capas y generalización

Los resultados anteriores se leen mejor como una descripción por capas de dónde sale el rendimiento de inferencia — tres capas, de abajo arriba:

- *Hardware y firmware:* el SoC, el subsistema de memoria y su techo de ancho de banda, el interconnect y los ajustes EEPROM (aquí, el intercalado de bancos bajos de SDRAM) — el techo físico.
- *Sistema operativo:* parámetros del kernel (governor, tamaño de página, sysctls de red, scheduler) y runtime (número de hilos, asignador, afinidad) — cuánto del techo alcanza la carga.
- *Software de aplicación:* el framework, sus kernels de cómputo, flags del compilador y algoritmos (fusión de operadores, cuantización, decodificación especulativa).

La regla que organiza todos los resultados de este informe — una reformulación de la visión roofline, no una idea nueva — es: *perfila el cuello de botella y optimiza la capa donde vive*. En el decode denso el cuello de botella es el ancho de banda de memoria, así que las ganancias viven en el hardware y el sistema operativo: el cuarto hilo mueve el decode un 16 % por sí solo, y el intercalado de firmware y un kernel más nuevo suben el suelo al 29 % sobre el récord previo, mientras que reescribir los kernels de aplicación no produce diferencia medible (Welch $p=0,59$, Sección 7.3). Los mismos kernels sí rinden en una carga compute-bound — 86 % en prefill, 15.2 % en decode MoE (Tabla 12) — y esperamos que el método, no la receta, se traslade a otro software de inferencia (serving en GPU, modelos de detección); validarlo es trabajo futuro.

Cuadro 12: Qué capa rinde, según el régimen de la carga.

Carga	Cuello de botella	Capa que rinde
Decode denso (este trabajo)	ancho de banda de memoria	hardware + sistema operativo
Decode MoE (complementario)	cómputo (3B activos/token)	software de aplicación
Prefill (ambos)	cómputo (matmuls por lotes)	software de aplicación

9 Conclusión

DeepSeek-R1-Distill-8B sostiene 8.32 tok/s en decode a contexto corto sobre cuatro placas Raspberry Pi 5 de 16 GB a cuatro hilos por nodo ($n=20$, IC 95 % [8.30,8.34]), bit-exact, la mejor cifra publicada para este modelo en este hardware con distributed-llama bajo restricciones bit-exact, y un 29 % por encima del récord previo. El decode no es constante, sin embargo: a partir del timing por-token cae casi linealmente al crecer el KV-cache — $t(L)=117+0,0174 L$ ms por token, $R^2=0,999$, independiente del prompt — hasta unos 5.0 tok/s a un contexto de 5,000 tokens, y el óptimo de cuatro hilos se mantiene en toda longitud de contexto. La atribución muestra que el resultado es ingeniería de sistema multicapa: el 29 % sobre el récord de 2024 es el efecto acumulado de todo el stack que construimos — distributed-llama con nuestros parches al framework y fusiones de kernel, tuning de kernel, scheduler y red de Linux, firmware SDRAM y cuatro hilos (las placas de 16 GB comparten el ancho de banda del récord de 8 GB y por tanto no aportan nada) — frente a un baseline más antiguo y sin tunear. En esta carga densa, los movers aislados son el número de hilos y el firmware; nuestras reescrituras de kernel de cómputo y el tuning de la NIC, que impulsan el MoE compute-bound del companion (+15.2 %) y el prefill aquí (+86 %), aportan ≈ 0 al decode denso limitado por ancho de banda. El decode denso está limitado por el ancho de banda de la LPDDR4X (10.1 de 17 GB/s por nodo, 87 % del tiempo por token en el nodo), y el contraste con el modelo Mixture-of-Experts *compute-bound*, donde el mismo software da 15 %, plantea el límite de ancho de banda como una comparación controlada. La vía *lossless* restante es emitir más tokens por lectura de pesos, lo que hace el decodificador especulativo adaptativo cuando la salida es predecible. Más allá de eso, el throughput de decode denso en este hardware es función del ancho de banda de memoria, y nuevas ganancias requieren cambios de hardware, como memoria más rápida, una interconexión más rápida o más nodos. Más en general, el rendimiento de inferencia es un stack de tres capas — hardware y firmware, sistema operativo, software de aplicación — y la optimización productiva es perfilar el cuello de botella y afinar la capa donde vive; en esta carga esa capa es el sistema y no los kernels, y esperamos que el método, aunque no la receta, se traslade a otro software de inferencia.

Limitaciones.

El óptimo de tres a cuatro hilos se invirtió entre una medición anterior y esta; el cambio coincidió con una actualización del kernel que no pudimos aislar, al no haber un downgrade seguro, así que lo atribuimos de forma tentativa. Y el headline es específico de la plataforma y la configuración: el 29 % sobre el récord previo es ingeniería de sistema — plataforma, configuración de hilos y firmware, e integración del upstream — y no reescritura de kernels. Ambas cosas son trabajo nuestro; solo la capa de sistema mueve una carga limitada por ancho de banda (Sección 7.3).

Referencias

- D. Correa Villa. *Pushing Four Raspberry Pis to the Memory Wall: Bit-Exact 30B Mixture-of-Experts Inference, 16 % over the Public Record*. 2026. DOI: 10.5281/zenodo.20357376.
- B. Tadych. *distributed-llama: Tensor-parallel LLM inference for home clusters*. <https://github.com/b4rtaz/distributed-llama>.
- DeepSeek R1 Distill 8B Q40 on 4×Raspberry Pi 5 8GB*. distributed-llama Discussion #162. <https://github.com/b4rtaz/distributed-llama/discussions/162>.
- Sseed Studio. *Distributed Inference of DeepSeek model on Raspberry Pi*. 2025.
- J. Geerling. *How is DeepSeek R1 on a Raspberry Pi?* 2025. <https://www.jeffgeerling.com/blog/2025/how-deepseek-r1-on-raspberry-pi>.

Z. Li et al. *prima.cpp: Distributed on-device LLM inference*. arXiv:2504.08791, 2025.

EXO Labs. *EXO: run your own AI cluster*. <https://github.com/exo-explore/exo>.

Y. Leviathan, M. Kalman, Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. arXiv:2211.17192.

Y. Fu et al. *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. arXiv:2402.02057.